

El Gato | Tic Tac Toe - Enunciado

// 03 MS Cliente – gato-client

PUERTOS :8081 / :8082 (INSTANCIAS)

Rol y principios del cliente

El cliente es la fachada orientada al jugador. Administra su identidad localmente, recibe el `serverUrl` destino en las operaciones de juego, y delega al server correspondiente. Mantiene un registro local de qué partida pertenece a qué server.

- Nunca retorna información de otros jugadores.
- Cada operación de juego que crea una partida o se une a una recibe el `serverUrl` en el request.
- Una vez registrada la asociación `gameId → serverUrl`, el jugador no necesita volver a indicar el server para esa partida.
- Aplica fallback cuando el server no responde.

03.1 Modelos de dominio (local)

PLAYER

- `id` — Long, generado
- `username` — String único
- `displayName` — String
- `createdAt` — LocalDateTime

GAMEREGISTRY

Registro local: mapea cada gameId al server donde fue creada.

- `id` — Long, generado
- `gameId` — Long (ID en el server)
- `serverUrl` — String (ej: `http://localhost:8080`)
- `playerId` — Long (jugador de este cliente)
- `registeredAt` — LocalDateTime

03.2 Manejo de fallos – Fallback

Obligatorio: el cliente no puede fallar si el server no responde

Toda comunicación del cliente hacia el server debe tener un timeout configurado y un mecanismo de fallback. Si el server no responde o retorna un error 5xx, el cliente **nunca debe propagar una excepción sin manejar**: retorna una respuesta estructurada indicando el problema.

TIMEOUT DE CONEXIÓN

El server no responde en el tiempo configurado (ej: 3 seg). El cliente retorna `503 Service Unavailable` con un mensaje claro.

ERROR DEL SERVER (5XX)

El server retorna un error interno. El cliente retorna `502 Bad Gateway` informando que el server destino tuvo un problema.

SERVER NO ENCONTRADO

El `serverUrl` ingresado no es alcanzable. El cliente retorna `503` antes de intentar registrar el gameId localmente.

EJEMPLO DE RESPUESTA DE FALLBACK

```
{
  "error": "SERVER_UNAVAILABLE",
  "serverUrl": "http://localhost:8080",
  "message": "El servidor de juego no está disponible. Intenta más tarde.",
  "timestamp": "2025-06-01T10:30:00"
}
```

Se recomienda implementar el fallback usando un bloque `try-catch` sobre la llamada HTTP en la clase `GatoServerClient`, capturando `ResourceAccessException` (timeout/conexión rechazada) y `HttpServerErrorException` (5xx).

03.3 Endpoints

Clic en cada endpoint para ver ejemplos.

GESTIÓN DE JUGADORES — /API/PLAYERS

POST

/api/players



Registra un nuevo jugador localmente en este cliente.

GET

/api/players/{playerId}



Datos del jugador. Solo puede consultarse a sí mismo (header X-Player-Id debe coincidir).

POST

/api/game/create



Crea una partida en el server indicado. El cliente inyecta el playerId y registra gameId→serverUrl localmente.

POST

/api/game/{gameId}/join



El jugador autenticado se une como O. Debe indicar el serverUrl donde está la partida.

POST

/api/game/{gameId}/move



Realiza una jugada. El cliente recupera el serverUrl del registro local por gameId — no se pide de nuevo.

GET

/api/game/{gameId}/state



Estado actual de la partida. El cliente consulta el serverUrl del registro local. Verifica que el jugador sea participante.

GET

/api/game/my-games



Todas las partidas registradas del jugador autenticado, agrupadas por serverUrl.

GET

/api/game/my-stats



Estadísticas del jugador consultadas en todos los servers registrados. Consolida wins/losses/draws.

← 02 MS Server

04 Estándares →

